

Welcome To Linux For You

User Name

Password:

The User Name field is required.

The Password field is required.

Figure 2: Validation check

- As explained in the view file source, the form will throw its data to the *check* function of the controller file. To access that data, use the built-in '*\$this->input->post('fieldname')*' API of the form helper. Here, they are stored in certain variables.
- Next, control will move to the model file, and it will check for the entered username and password in the database. If the count returns more than one, the username and password is correct, and '*Login Successful*' will be displayed, else '*Wrong user name/password*'. Submitting a form with blank fields yields validation errors, like those in Figure 2.

File loginmodel.php

```
<?php
class LoginModel extends CI_Model {
function check($userName,$password)
{
$query="select count('userName') as count from lfy_users where
userName='$userName' and password='$password'";
$result=$this->db->query($query);
return $result->result_array();
}
}
?>
```

The *check()* function will check for the user name and password in the database, as described above.



Note: I have used a simple *varchar* password field in this example, to keep things simple. You should encrypt the password before storing, using MD5, SHA1 types.

Pagination

Display grids are the simplest way of representing data. But what if there are thousands of records? We use a pagination technique to display a certain amount of records on every page. The built-in CodeIgniter pagination APIs make this quite easy. Here's an example displaying data of

S.No	User Name	Password
5	CodeIgniter	ci2
6	Display Demo	display

Figure 3: Pagination

seven users, paginating to two records per page (*/var/www/ci2/application/controllers/displaycontroller.php*).

File displaycontroller.php

```
<?php
class DisplayController extends CI_Controller {
function display($offset=0)
{
$limit=2; #1
$this->load->model('displaymodel');
$result['contents']=$this->displaymodel->getRows($limit,$offset);
$result['numOfRows']=$this->displaymodel->countRows();
$this->load->library('pagination'); #2
$this->load->library('table');
$config=array(
'base_url'=>'http://localhost/ci2/index.php/
displaycontroller/display',
'total_rows'=>$result['numOfRows'][0]['count'],
'per_page'=>$limit,
'num_links'=>2
); #3
$this->pagination->initialize($config); #4
$result['pagination']=$this->pagination->create_links();
#5
$this->load->view('displayview',$result);
}
}
?>
```

Here's the explanation:

- The idea is to fetch the records according to the limit set (this will also be record_per_page) and set the offset value in the URI segment. The initial offset value is 0.
- We loaded the pagination library.
- Pagination demands certain parameters to be initialised—we have created an array of such parameters. '*base_url*' is the link to which the page is to be reloaded after getting a hit on any page number; '*total_rows*' refers to the total records; '*per_page*' is the limit of records to show on one page; '*num_links*' is the maximum links allowed for display (after that, '*First*' and '*Last*' will appear).
- We initialised pagination with the above parameters.
- We fetched the created links into the pagination variable. You will see something like what is shown in Figure 3